Cây nhị phân tìm kiếm

- **Điểm nổi bật**
  - **Các khóa trong cây con trái < khóa của gốc < Các khóa trong cây con phải**
  - **Cây con trái, phải đều là BST**
  - **Khóa là duy nhất trong cây**
  - **Kỹ thuật lập trình thích hợp: đệ qui**
  - **Tác vụ được hỗ trợ tốt: tìm kiếm, thêm, xóa; không tốt: duyệt???**
  - **Với cây có N nút thì độ cao của cây: logN <= h <= N**
    - Trường hợp "đẹp nhất"
    - Trường hợp suy biến
- **Thư viện C++**
  - **set**
  - **map**
- **Các biến thể**
  - **Cây nhị phân tìm kiếm cân bằng hoàn toàn - CBT**
    - Là BST có:
      - Cây con trái và phải đều là CBT
      - Số nút của 2 cây con lệch nhau không quá 1
    - Chiều cao của cây có N nút: hCBT = ???
  - **Cây AVL**
    - Là BST có:
      - Cây con trái và phải đều là AVL
      - Độ cao của 2 cây con lệch nhau không quá 1
    - Chiều cao của cây có N nút: h = ???
      - hAVL < 145% hCBT
  - **Cây đỏ đen**
    - Là BST có:
      - Các cạnh được tô màu ĐỎ hoặc ĐEN
      - Trên mỗi đường đi từ gốc đến lá không có 2 canh ĐỎ kề nhau
      - Cạnh nối với nút ngoài (NULL) luôn có màu đen
    - Các đường đi từ gốc đến mỗi nút ngoài đều có số cạnh đen bằng nhau --> B: chiều cao đen của cây
      - $2^B \le N + 1 \le 4^B$
      - $\frac{1}{2}\log(N+1) \le B \le \log(N+1)$
      - $\log(N+1) \le H \le 2\log(N+1)$
- **Các thao tác**
  - **Khởi tạo**
  - **Kiểm tra rỗng**
  - **Tìm kiếm trên cây**
  - **Thêm một phần tử**
    - Nút mới thêm vào ở vị trí của 1 "nút ngoài"
  - **Xóa một phần tử p**
    - Nút có ít hơn 2 con
    - Nút có đủ 2 con --> Tìm phần tử thế mạng q
      - q là trái nhất trên cây con phải hoặc phải nhất trên cây con trái của p
      - Có thể xóa bằng cách lưu giữ dữ liệu của q về chỗ của p và xóa q HOẶC tổ chức lại cây để q đảm nhận vai trò của p và xóa p
  - **Duyệt cây**
    - Thứ tự trước: nLR
    - Thứ tự giữa: LnR
    - Thứ tự sau: LRn
  - **Hủy cây**
    - cần duyệt theo thứ tự sau để hủy
- **Kiểu dữ liệu thường được dùng để cài đặt liên kết**
  - **Con trỏ**
    - Mỗi nút sử dùng 2 con trỏ để lưu địa chỉ 2 cây con
    - Cài đặt phức tạp, tính linh hoạt cao
  - **Mảng một chiều**
    - Mỗi nút sử dụng 2 số nguyên là địa chỉ (tương đối) của 2 nút gốc cây con
    - Cần thêm 1 biến phụ để cho biết địa chỉ (tương đối) của nút sẽ được thêm mới
    - Tính linh hoạt bị hạn chế